

SkyBase: An Intelligent Airline Management Database System

Table of Contents

Table of Contents	1
1. Introduction and Sector Selection.....	2
2. Part 1: Database Design and Implementation	2
2.1 Task A: Requirements Analysis	2
2.1.1 Background and Problem Statement.....	2
2.1.2 Functional Requirements.....	2
2.1.3 Non-Functional Requirements.....	3
2.1.4 Entity Identification	4
2.1.5 Data Dictionary.....	5
2.2 Task B: Entity Relationship Diagrams	9
2.2.1 Conceptual ERD	9
2.2.2 Logical ERD	9
2.2.3 Physical ERD	10
2.2.4 Cardinality Summary	11
2.3 Task C: Normalisation to Third Normal Form.....	12
2.3.1 Unnormalised Form (UNF)	12
2.3.2 First Normal Form (1NF)	12
2.3.3 Second Normal Form (2NF)	13
2.3.4 Third Normal Form (3NF)	13
2.4 Task D: Database Implementation	14
2.5 Task E: Data Population.....	16
2.6 Task F: SQL Queries.....	17
Query 1: Recent Bookings (4-table JOIN)	17
Query 2: Flight Occupancy (4-table JOIN with aggregation).....	17
Query 3: Route Revenue (4-table JOIN with SUM)	18
Query 4: Crew Workload (2-table JOIN with COUNT)	19
Query 5: Full Booking History (5-table JOIN).....	19
2.7 Task G: CRUD Application Interface	20
3. Part 2: Research and Development.....	22

3.1 Big Data.....	22
3.2 Hadoop and Relational Database Systems.....	22
3.3 Relational, NoSQL and NewSQL Databases	23
4. References.....	23

1. Introduction and Sector Selection

Diamond Group manages flight operations across more than thirty routes from London, with scheduling, booking and crew data held in separate systems that share no data layer. Generating a revenue figure requires manual exports and reconciliation across all three. SkyBase replaces this with a normalised PostgreSQL relational database and a Python/Flask web application covering the full operational stack from passenger booking through to crew assignment and baggage handling.

The chosen sector is transport, specifically commercial aviation. The domain suits a relational approach well: records are highly structured, relationships between entities are fixed and well understood, and data integrity is non-negotiable. A duplicate booking or an orphaned payment record has immediate operational consequences that a properly designed database prevents.

2. Part 1: Database Design and Implementation

2.1 Task A: Requirements Analysis

2.1.1 Background and Problem Statement

Diamond Group's current setup stores flight schedules, passenger bookings and crew rosters across three separate systems with no shared data layer. Producing an occupancy report requires exporting from all three and reconciling the results by hand, introducing delays and the risk of human error. There is also a compliance concern: passenger data including passport numbers and payment details is held in unprotected spreadsheets, creating a GDPR exposure that a centralised access-controlled database resolves.

SkyBase centralises all operational data in a single PostgreSQL database accessible through a Flask web interface. Foreign key constraints enforce referential integrity at the database level, parameterised queries prevent SQL injection throughout the application, and the schema is normalised to third normal form to eliminate update anomalies. Staff can search, update and report through a browser without writing any SQL.

2.1.2 Functional Requirements

Table 1 lists the ten functional requirements identified for the SkyBase system.

ID	Requirement	Description
FR-01	Passenger Management	Create, read, update and delete passenger records including personal details, contact information and passport data.
FR-02	Flight Management	Manage scheduled flights including departure and arrival airports, assigned aircraft, times, status and base price.
FR-03	Booking Management	Create and manage bookings linking a passenger to a specific flight with seat class and booking status.
FR-04	Payment Recording	Record payment transactions per booking covering method, amount, currency and status.
ID	Requirement	Description
FR-05	Crew Assignment	Assign crew members to flights with a designated role and maintain a traceable assignment record.
FR-06	Baggage Tracking	Register baggage items against bookings with tag number, weight, type and handling status.
FR-07	Seat Management	Define seats per aircraft with class, window and aisle flags. Seat allocation is optional at time of booking.
FR-08	Operational Reporting	Generate reports on route revenue, flight occupancy, crew workload and payment breakdown via SQL aggregation.
FR-09	Passenger Search	Search passenger records by name or email using caseinsensitive pattern matching.
FR-10	Dashboard	Display entity row counts and a live feed of recent bookings on the main landing page.

2.1.3 Non-Functional Requirements

Table 2 lists the non-functional requirements governing system behaviour and quality attributes.

ID	Category	Description
NFR-01	Performance	Standard CRUD operations complete within 2 seconds under normal load.
NFR-02	Security	All queries use parameterised inputs. No raw string concatenation in SQL.
NFR-03	Data Integrity	Foreign key, NOT NULL and UNIQUE constraints are enforced at the database level.

NFR-04	GDPR	Passenger personal data can be fully deleted on request without leaving orphaned records.
NFR-05	Scalability	The schema supports growth to 500,000 passenger records and 50,000 flights without structural changes.
NFR-06	Usability	All operations are available through browser forms. No SQL knowledge required from staff.
NFR-07	Maintainability	All tables follow snake_case naming conventions and are documented in the data dictionary.

2.1.4 Entity Identification

Ten entities were identified from analysis of Diamond Group's operational requirements. Table 3 lists each entity with a brief description and its primary relationships within the schema.

#	Entity	Description	Key Relationship
1	passenger	Personal, contact and passport details for individuals booking flights.	1:M with booking
#	Entity	Description	Key Relationship
2	flight	A scheduled service between two airports at a defined time.	1:M with booking and crew_assignment
3	airport	A departure or arrival location identified by IATA code.	Referenced twice in flight (dep/arr)
4	aircraft	A specific aircraft in the fleet with model and seat capacity.	1:M with flight and seat
5	booking	Links one passenger to one flight with seat class and status.	FK to passenger, flight and seat
6	payment	The financial transaction associated with a booking.	1:1 with booking
7	seat	An individual seat on an aircraft.	Optional FK on booking
8	baggage	Luggage items registered against a booking.	M:1 with booking
9	crew	Staff members eligible for flight duties.	1:M with crew_assignment
10	crew_assignment	Junction entity resolving the many-to-many between crew and flight.	FK to crew and flight

2.1.5 Data Dictionary

The following tables document every attribute across all ten entities, specifying the data type, constraint and purpose of each field as implemented in PostgreSQL 18.

passenger

Attribute	Data Type	Constraint	Description
passenger_id	SERIAL	PK	Unique identifier, autoincremented
first_name	VARCHAR(50)	NOT NULL	Given name
last_name	VARCHAR(50)	NOT NULL	Family name
email	VARCHAR(100)	NOT NULL, UNIQUE	Contact email, unique per record
phone	VARCHAR(20)	NOT NULL	Contact telephone number
date_of_birth	DATE	NOT NULL	Date of birth for identity verification
passport_number	VARCHAR(20)	NULL	Passport number for international travel
nationality	VARCHAR(50)	NULL	Country of nationality
gender	VARCHAR(10)	NULL	Passenger gender
Attribute	Data Type	Constraint	Description
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Record creation timestamp

flight

Attribute	Data Type	Constraint	Description
flight_id	SERIAL	PK	Unique identifier
flight_number	VARCHAR(10)	NOT NULL	Airline flight code, e.g. SK101
departure_airport_id	INTEGER	FK to airport	Departure airport reference
arrival_airport_id	INTEGER	FK to airport	Arrival airport reference
aircraft_id	INTEGER	FK to aircraft	Operating aircraft reference

departure_time	TIMESTAMP	NOT NULL	Scheduled departure time in UTC
arrival_time	TIMESTAMP	NOT NULL	Scheduled arrival time in UTC
status	VARCHAR (20)	NOT NULL	Scheduled, Departed, Landed or Cancelled
base_price	NUMERIC (10, 2)	NOT NULL	Economy base fare in GBP

airport

Attribute	Data Type	Constraint	Description
airport_id	SERIAL	PK	Unique identifier
iata_code	CHAR (3)	NOT NULL, UNIQUE	Three-letter IATA airport code
name	VARCHAR (100)	NOT NULL	Full official airport name
city	VARCHAR (50)	NOT NULL	City location
country	VARCHAR (50)	NOT NULL	Country location
timezone	VARCHAR (50)	NOT NULL	IANA timezone identifier

aircraft

Attribute	Data Type	Constraint	Description
aircraft_id	SERIAL	PK	Unique identifier
registration_number	VARCHAR (15)	NOT NULL, UNIQUE	Official tail number
model	VARCHAR (50)	NOT NULL	Aircraft model designation
manufacturer	VARCHAR (50)	NOT NULL	Manufacturer name
total_seats	INTEGER	NOT NULL	Total passenger seat capacity
year_manufactured	INTEGER	NOT NULL	Year of manufacture
Attribute	Data Type	Constraint	Description
status	VARCHAR (20)	NOT NULL	Active, Maintenance or Retired

seat

Attribute	Data Type	Constraint	Description
-----------	-----------	------------	-------------

seat_id	SERIAL	PK	Unique identifier
aircraft_id	INTEGER	FK to aircraft	Aircraft this seat belongs to
seat_number	VARCHAR (5)	NOT NULL	Seat code, e.g. 12A
seat_class	VARCHAR (20)	NOT NULL	Economy, Business or First
is_window	BOOLEAN	NOT NULL	True if the seat is adjacent to a window
is_aisle	BOOLEAN	NOT NULL	True if the seat is on the aisle

booking

Attribute	Data Type	Constraint	Description
booking_id	SERIAL	PK	Unique booking reference
passenger_id	INTEGER	FK to passenger	Passenger who made the booking
flight_id	INTEGER	FK to flight	Flight being booked
seat_id	INTEGER	FK to seat, NULLABLE	Seat assigned at check-in (optional)
booking_date	TIMESTAMP	NOT NULL, DEFAULT NOW()	Timestamp when the booking was created
seat_class	VARCHAR (20)	NOT NULL	Economy, Business or First
status	VARCHAR (20)	NOT NULL	Confirmed, Cancelled or Pending

payment

Attribute	Data Type	Constraint	Description
payment_id	SERIAL	PK	Unique payment identifier
booking_id	INTEGER	FK to booking	Booking this payment covers
amount	NUMERIC (10, 2)	NOT NULL	Transaction amount
currency	CHAR (3)	NOT NULL	ISO 4217 currency code, e.g. GBP
payment_method	VARCHAR (30)	NOT NULL	Credit Card, Debit Card or PayPal

payment_date	TIMESTAMP	NOT NULL	Timestamp when payment was processed
Attribute	Data Type	Constraint	Description
status	VARCHAR (20)	NOT NULL	Completed, Failed or Refunded

baggage

Attribute	Data Type	Constraint	Description
baggage_id	SERIAL	PK	Unique baggage record identifier
booking_id	INTEGER	FK to booking	Associated booking
tag_number	VARCHAR (20)	NOT NULL, UNIQUE	Barcode tag assigned at checkin
weight_kg	NUMERIC (5, 2)	NOT NULL	Declared weight in kilograms
baggage_type	VARCHAR (20)	NOT NULL	Checked, Carry-on, Sports or Fragile
status	VARCHAR (20)	NOT NULL	Checked-in, Loaded, Delivered or Lost

crew

Attribute	Data Type	Constraint	Description
crew_id	SERIAL	PK	Unique identifier
first_name	VARCHAR (50)	NOT NULL	Given name
last_name	VARCHAR (50)	NOT NULL	Family name
email	VARCHAR (100)	NOT NULL	Contact email address
phone	VARCHAR (20)	NOT NULL	Contact telephone number
role	VARCHAR (50)	NOT NULL	Captain, First Officer or Cabin Crew
licence_number	VARCHAR (20)	NULL	ATPL or CPL licence number (pilots only)
date_of_joining	DATE	NOT NULL	Start date with the airline

crew_assignment

Attribute	Data Type	Constraint	Description
-----------	-----------	------------	-------------

assignment_id	SERIAL	PK	Unique assignment record
crew_id	INTEGER	FK to crew	Crew member assigned
flight_id	INTEGER	FK to flight	Flight being staffed
role_on_flight	VARCHAR (50)	NOT NULL	Specific role on this particular flight

2.2 Task B: Entity Relationship Diagrams

2.2.1 Conceptual ERD

The conceptual ERD maps all ten entities and their relationships without specifying data types or implementation detail. Booking is the central transactional entity, linking Passenger to Flight. Each passenger may hold multiple bookings, and each flight may receive bookings from many passengers. A Flight references Airport twice under separate foreign keys for departure and arrival. CrewAssignment acts as a junction entity between Crew and Flight, resolving what would otherwise be a direct many-to-many relationship. Each Booking has one Payment record, an optional Seat reference, and may have multiple Baggage items attached.

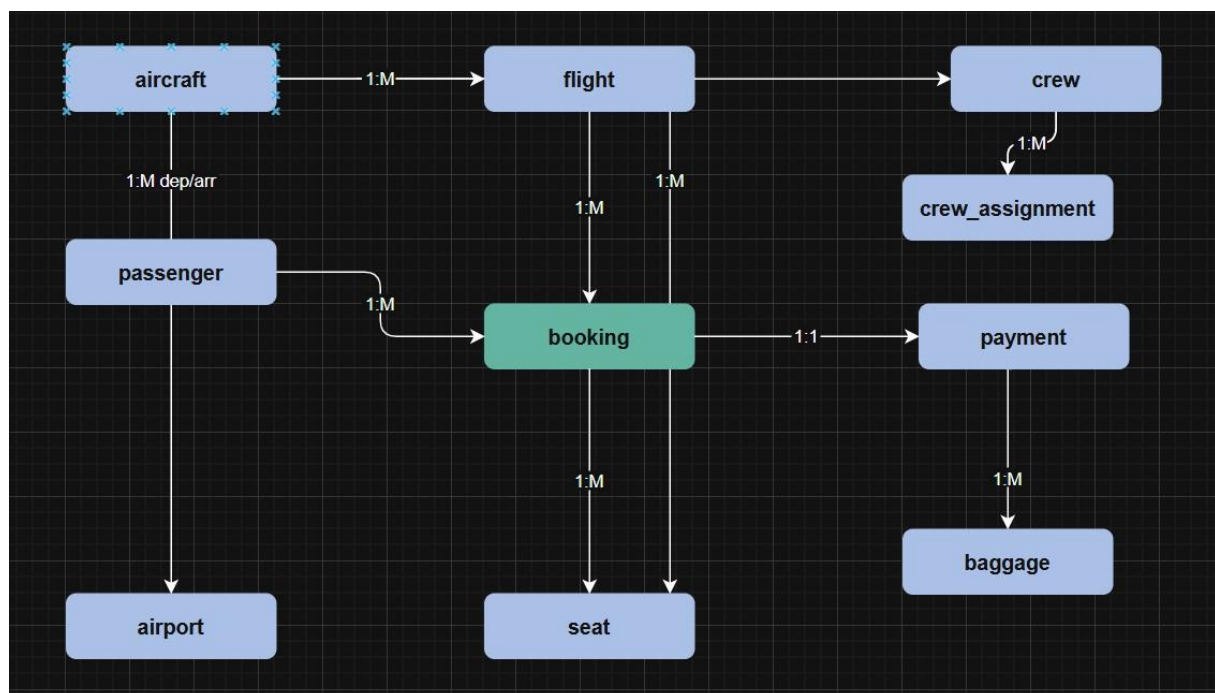


Figure 1: Conceptual ERD - entities and relationships, no implementation detail

2.2.2 Logical ERD

The logical ERD adds attributes, primary keys, foreign keys and Crow's Foot cardinality notation. Every entity uses a SERIAL surrogate primary key to avoid dependency on natural keys that may change over time. The two Airport foreign keys on Flight are named departure_airport_id and arrival_airport_id to make their roles explicit. The seat_id on

Booking is nullable because seat assignment happens at check-in rather than at the point of booking, reflecting a real operational rule.

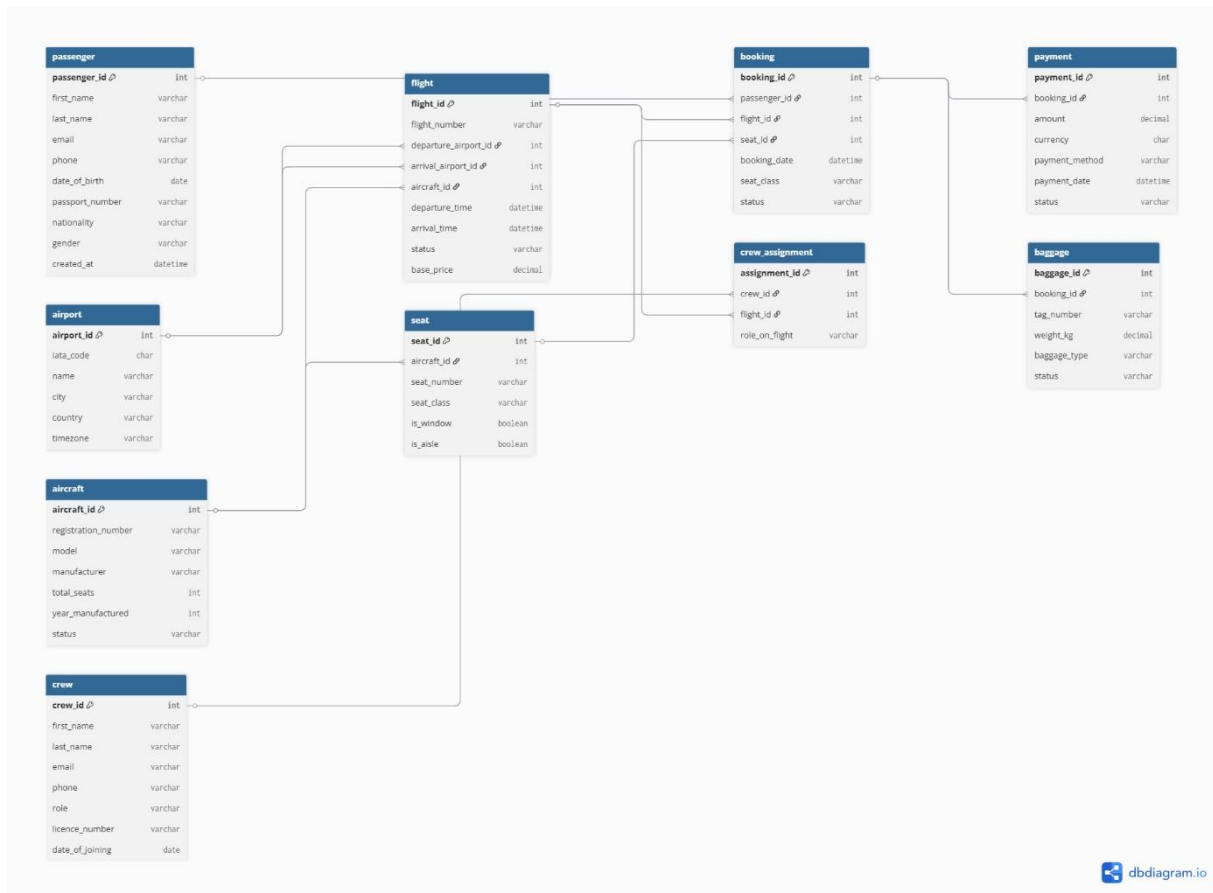


Figure 2: Logical ERD - attributes, primary and foreign keys, Crow's Foot cardinality

2.2.3 Physical ERD

The physical ERD uses exact PostgreSQL data types. NUMERIC replaces FLOAT for monetary amounts to avoid floating-point rounding errors on payment calculations. CHAR(3) is used for IATA airport codes, which are always exactly three characters. BOOLEAN covers the is_window and is_aisle flags on the Seat entity. VARCHAR lengths are set conservatively based on real-world domain constraints. All NOT NULL and UNIQUE constraints are shown as they appear in the live database.

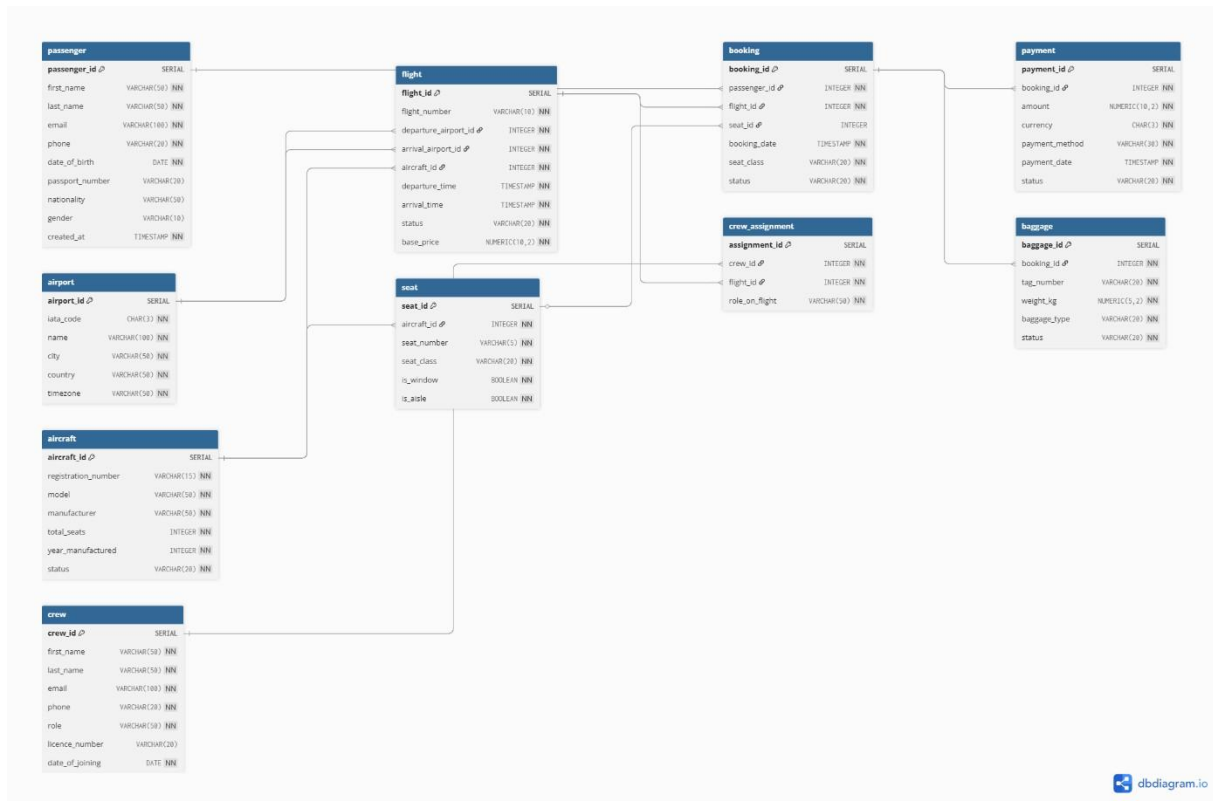


Figure 3: Physical ERD - exact PostgreSQL data types and constraints as implemented

2.2.4 Cardinality Summary

Table 4 summarises all relationships and their cardinality types across the ten-entity schema.

Relationship	Type	Description
Passenger to Booking	1:M	One passenger may have many bookings.
Flight to Booking	1:M	One flight receives many bookings.
Booking to Payment	1:1	Each booking has exactly one payment record.
Booking to Baggage	1:M	One booking can have multiple baggage items.
Seat to Booking	1:M (nullable)	One seat may be optionally referenced across many bookings.
Airport to Flight (dep/arr)	1:M	One airport serves as departure or arrival for many flights.
Aircraft to Flight	1:M	One aircraft operates many flights.
Aircraft to Seat	1:M	One aircraft has many seats.
Crew to Crew Assignment	1:M	One crew member has many flight assignments.

Flight to Crew Assignment	1:M	One flight has many crew members assigned.
---------------------------	-----	--

2.3 Task C: Normalisation to Third Normal Form

Normalisation is the process of structuring a relational schema to remove data redundancy and prevent anomalies that cause inconsistencies on insert, update or delete (Codd, 1970). The steps below trace the SkyBase design from a flat unnormalised booking record through to the fully decomposed third normal form schema used in the live database.

2.3.1 Unnormalised Form (UNF)

In a flat-table design all booking data sits in one relation. Table 5 shows the core problems: crew names and bag tags appear as comma-separated lists within single cells, passenger details repeat across rows, and flight data is duplicated wherever the same flight appears more than once.

BookingRef	Passenger	Email	FlightNo	Crew (multivalued)	Baggage (multivalued)
B001	John Smith	john@mail.com	DG-1042	Brown, Davis, Jones	TAG001 23.5kg, TAG002 18.2kg
B002	John Smith	john@mail.com	DG-2087	Wilson, Clark	TAG003 20.0kg
B003	Sarah Lee	sarah@mail.com	DG-1042	Brown, Davis, Jones	TAG004 25kg, TAG005 12kg

Updating Smith's email requires locating every row where his name appears. Deleting booking B003 removes the only record of bag tags TAG004 and TAG005. Both are classic anomalies that normalisation eliminates.

2.3.2 First Normal Form (1NF)

First normal form requires atomic values in every cell and no repeating groups (Date, 2004). The crew and baggage columns are decomposed so each row holds exactly one crew member and one bag tag. A composite key of (BookingRef, CrewName, BagTag) uniquely identifies each row. The route field is split into separate departure and arrival columns. Figure 4 shows the resulting 1NF table.

```
AIRLINE_1NF(BookingRef*, PassengerName, Email, Passport, FlightNo,
            DepAirport, ArrAirport, AircraftReg, SeatNo, SeatClass,
            PayAmount, PayMethod, CrewName*, CrewRole,
            BagTag*, BagWeight, BagType)

* = component of composite primary key
```

Figure 4: 1NF - multi-valued columns decomposed, composite primary key applied

Atomicity is now satisfied throughout, but passenger and flight data repeats more heavily across rows. Second normal form addresses this.

2.3.3 Second Normal Form (2NF)

Second normal form requires every non-key attribute to depend on the complete primary key, not just part of it. Passenger name, email and passport depend only on BookingRef. Flight details depend only on FlightNo. Baggage weight and type depend only on BagTag, and crew role depends only on CrewName. Removing these partial dependencies produces five separate relations as shown in Figure 5.

```
BOOKING(BookingRef*, PassengerName, Email, Passport, FlightNo,
        SeatNo, SeatClass, PayAmount, PayMethod)
FLIGHT(FlightNo*, DepAirport, ArrAirport, AircraftReg,
        DepTime, ArrTime, Status, BasePrice)
BAGGAGE(BagTag*, BookingRef, BagWeight, BagType, BagStatus)
CREW(CrewName*, CrewRole, LicenceNo, Phone)
CREW_FLIGHT(CrewName*, FlightNo*, RoleOnFlight)
```

Figure

5: 2NF - partial dependencies removed, five separate relations produced

2.3.4 Third Normal Form (3NF)

Third normal form removes transitive dependencies, where a non-key attribute determines another non-key attribute (Codd, 1970). In BOOKING, passenger name determines email and passport number, so a PASSENGER table is extracted and BOOKING stores passenger_id as a foreign key instead. Payment amount and method describe a financial transaction rather than the booking itself, so PAYMENT is extracted with its own primary key. Seat number and class describe a physical seat on an aircraft, so SEAT is extracted and BOOKING carries a nullable seat_id. In FLIGHT, airport codes determine city and country, so AIRPORT is extracted. Aircraft registration determines model and manufacturer, so AIRCRAFT is extracted. The resulting ten-table schema is shown in Figure 6.

```
passenger(passenger_id*, first_name, last_name, email, phone,
          date_of_birth, passport_number, nationality, gender, created_at)
aircraft(aircraft_id*, registration_number, model, manufacturer,
         total_seats, year_manufactured, status)
airport(airport_id*, iata_code, name, city, country, timezone)
flight(flight_id*, flight_number, departure_airport_id+,
       arrival_airport_id+, aircraft_id+, departure_time,
       arrival_time, status, base_price)
seat(seat_id*, aircraft_id+, seat_number, seat_class,
     is_window, is_aisle)
crew(crew_id*, first_name, last_name, email, phone,
     role, licence_number, date_of_joining)
booking(booking_id*, passenger_id+, flight_id+, seat_id+,
       booking_date, seat_class, status)
payment(payment_id*, booking_id+, amount, currency,
       payment_method, payment_date, status)
baggage(baggage_id+, booking_id+, tag_number, weight_kg,
       baggage_type, status)
crew_assignment(assignment_id*, crew_id+, flight_id+, role_on_flight)

* = Primary Key    + = Foreign Key
```

Figure 6: 3NF - ten-table schema, all transitive dependencies resolved

Every non-key attribute in each relation now depends directly and solely on that relation's primary key. No insert, update or delete anomalies remain in the design, and the schema maps directly to the SkyBase PostgreSQL implementation.

2.4 Task D: Database Implementation

All ten tables and the passenger_bookings view were created in PostgreSQL 18 using pgAdmin 4. Figure 7 shows the pgAdmin object tree confirming all tables and the view are present under the public schema. Figures 8 to 10 show the CREATE TABLE scripts for the passenger, flight and booking tables, demonstrating the use of primary keys, foreign keys, NOT NULL and UNIQUE constraints. The booking table carries three foreign keys, to passenger, flight and a nullable seat reference, which reflects the business rule that seat assignment is optional at booking time.

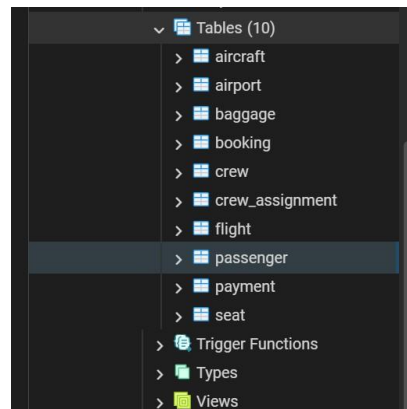


Figure 7: pgAdmin object tree - all 10 tables and passenger_bookings view confirmed

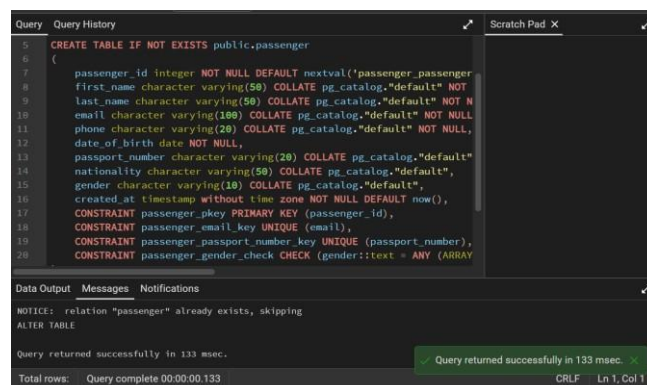


Figure 8: CREATE TABLE passenger - primary key, NOT NULL and UNIQUE constraints

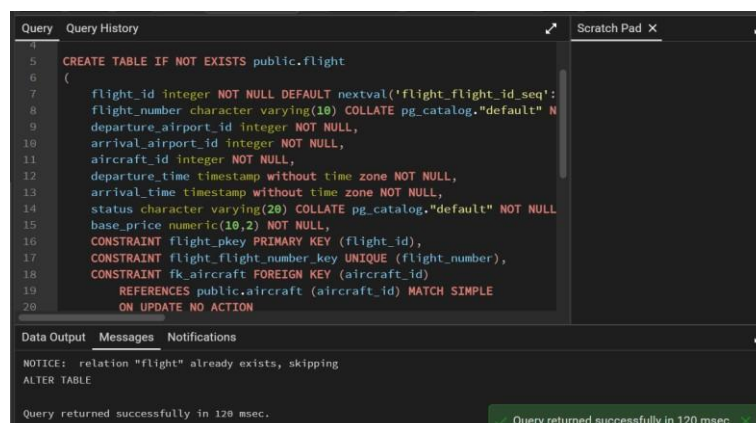


Figure 9: CREATE TABLE flight - primary key and foreign keys to airport and aircraft


```

5 CREATE TABLE IF NOT EXISTS public.flight
6 (
7     flight_id integer NOT NULL DEFAULT nextval('flight_flight_id_seq'),
8     flight_number character varying(10) COLLATE pg_catalog."default" NOT NULL,
9     departure_airport_id integer NOT NULL,
10    arrival_airport_id integer NOT NULL,
11    aircraft_id integer NOT NULL,
12    departure_time timestamp without time zone NOT NULL,
13    arrival_time timestamp without time zone NOT NULL,
14    status character varying(20) COLLATE pg_catalog."default" NOT NULL,
15    base_price numeric(10,2) NOT NULL,
16    CONSTRAINT flight_pkey PRIMARY KEY (flight_id),
17    CONSTRAINT flight_flight_number_key UNIQUE (flight_number),
18    CONSTRAINT fk_aircraft FOREIGN KEY (aircraft_id)

```

NOTICE: relation "flight" already exists, skipping
ALTER TABLE

Query returned successfully in 115 msec.

✓ Query returned successfully in 115 msec. ✕

Figure 10: CREATE TABLE booking - three foreign keys including nullable seat_id

The passenger_bookings view joins passenger, booking and flight to provide a ready-made reporting surface. Rather than requiring staff to write join queries manually, the view exposes a clean result set that can be queried with a simple SELECT. Its definition is shown below.

```

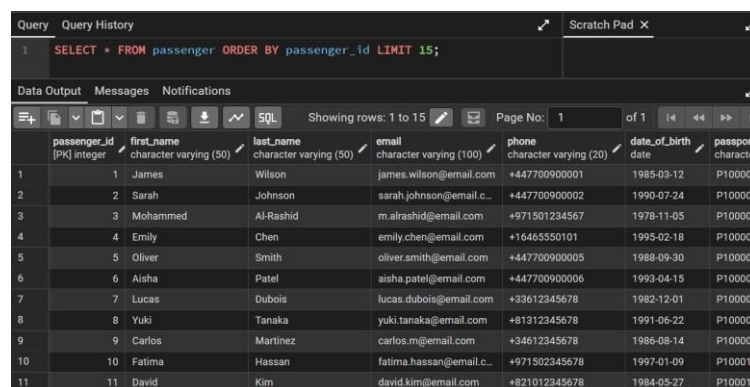
CREATE OR REPLACE VIEW passenger_bookings AS
SELECT
    p.passenger_id,
    p.first_name || ' ' || p.last_name AS passenger_name,
    p.email,
    b.booking_id,
    b.booking_date,
    b.seat_class,
    b.status AS booking_status,
    f.flight_number,
    f.departure_time,
    f.arrival_time
FROM passenger p
JOIN booking b ON b.passenger_id = p.passenger_id
JOIN flight f ON f.flight_id = b.flight_id;

```

2.5 Task E: Data Population

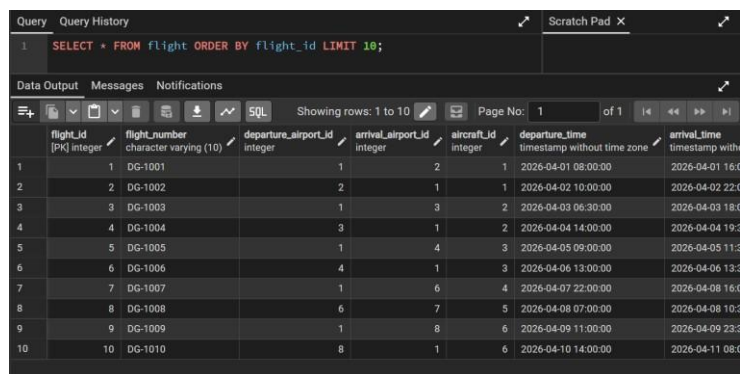
All ten tables were populated using INSERT statements executed in pgAdmin. Passenger records include realistic names, contact details, nationalities and passport numbers. Flight records use real IATA codes for departure and arrival airports with plausible timestamps and base prices. Each booking links a passenger to a flight with a seat class and status, and a corresponding payment record is inserted alongside it. Baggage records, seat definitions and crew assignments are also populated across all relevant tables.

Figure 11 shows the passenger table. Figure 12 shows the flight and booking tables with realistic data across all columns. Figure 13 shows a UNION query that counts rows across all ten tables, confirming that each has been populated with data. This approach to population, writing and executing INSERT statements directly in SQL rather than importing from a CSV file, demonstrates direct command of the data manipulation language and ensures the data model functions correctly end to end.



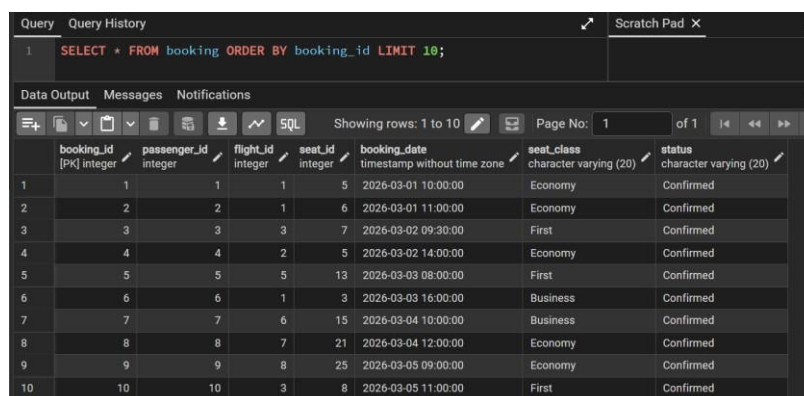
	passenger_id [PK] integer	first_name character varying (50)	last_name character varying (50)	email character varying (100)	phone character varying (20)	date_of_birth date	passport character
1	1	James	Wilson	james.wilson@email.com	+447700900001	1985-03-12	P100001
2	2	Sarah	Johnson	sarah.johnson@email.c...	+447700900002	1990-07-24	P100002
3	3	Mohammed	Al-Rashid	m.alrashid@email.com	+971501234567	1978-11-05	P100003
4	4	Emily	Chen	emily.chen@email.com	+16465550101	1995-02-18	P100004
5	5	Oliver	Smith	oliver.smith@email.com	+447700900005	1988-09-30	P100005
6	6	Aisha	Patel	aisha.patel@email.com	+447700900006	1993-04-15	P100006
7	7	Lucas	Dubois	lucas.dubois@email.com	+33612345678	1982-12-01	P100007
8	8	Yuki	Tanaka	yuki.tanaka@email.com	+81312345678	1991-06-22	P100008
9	9	Carlos	Martinez	carlos.m@email.com	+34612345678	1986-08-14	P100009
10	10	Fatima	Hassan	fatima.hassan@email.c...	+971502345678	1997-01-09	P100010
11	11	David	Kim	david.kim@email.com	+821012345678	1984-05-27	P100011

Figure 11: SELECT * FROM passenger - realistic data across all columns



	flight_id [PK] integer	flight_number character varying (10)	departure_airport_id integer	arrival_airport_id integer	aircraft_id integer	departure_time timestamp without time zone	arrival_time timestamp without time zone
1	1	DG-1001	1	2	1	2026-04-01 08:00:00	2026-04-01 16:00:00
2	2	DG-1002	2	1	1	2026-04-02 10:00:00	2026-04-02 22:00:00
3	3	DG-1003	1	3	2	2026-04-03 06:30:00	2026-04-03 18:00:00
4	4	DG-1004	3	1	2	2026-04-04 14:00:00	2026-04-04 19:00:00
5	5	DG-1005	1	4	3	2026-04-05 09:00:00	2026-04-05 11:30:00
6	6	DG-1006	4	1	3	2026-04-06 13:00:00	2026-04-06 13:30:00
7	7	DG-1007	1	6	4	2026-04-07 22:00:00	2026-04-08 16:00:00
8	8	DG-1008	6	7	5	2026-04-08 07:00:00	2026-04-08 10:30:00
9	9	DG-1009	1	8	6	2026-04-09 11:00:00	2026-04-09 23:30:00
10	10	DG-1010	8	1	6	2026-04-10 14:00:00	2026-04-11 08:00:00

Figure 12: Flight and booking tables populated with realistic records



	booking_id [PK] integer	passenger_id integer	flight_id integer	seat_id integer	booking_date timestamp without time zone	seat_class character varying (20)	status character varying (20)
1	1	1	1	5	2026-03-01 10:00:00	Economy	Confirmed
2	2	2	1	6	2026-03-01 11:00:00	Economy	Confirmed
3	3	3	3	7	2026-03-02 09:30:00	First	Confirmed
4	4	4	2	5	2026-03-02 14:00:00	Economy	Confirmed
5	5	5	5	13	2026-03-03 08:00:00	First	Confirmed
6	6	6	6	3	2026-03-03 16:00:00	Business	Confirmed
7	7	7	6	15	2026-03-04 10:00:00	Business	Confirmed
8	8	8	7	21	2026-03-04 12:00:00	Economy	Confirmed
9	9	9	9	25	2026-03-05 09:00:00	Economy	Confirmed
10	10	10	10	3	2026-03-05 11:00:00	First	Confirmed

Figure 13: UNION row count query - all ten tables confirmed populated

2.6 Task F: SQL Queries

Five SQL queries demonstrate the relational capability of the schema. Each joins multiple tables and returns results that could not be retrieved from any single table in isolation. The queries increase in complexity from a four-table join through to a five-table join with aggregated payment data. They were written directly in SQL and also serve as the Text-to-SQL layer powering the Flask reports page, where the application translates user interactions into structured queries against the live database.

Query 1: Recent Bookings (4-table JOIN)

Joins booking, passenger, flight and airport. The airport table is aliased twice to produce a formatted departure-to-arrival route string in a single column. Results are ordered by booking ID descending so the most recent activity appears first.

```
SELECT b.booking_id,
       p.first_name || ' ' || p.last_name AS passenger,
       f.flight_number,
       dep.iata_code || ' > ' || arr.iata_code AS route,
       b.seat_class,
       b.status
FROM   booking b
JOIN   passenger p ON b.passenger_id = p.passenger_id
JOIN   flight f ON b.flight_id = f.flight_id
JOIN   airport dep ON f.departure_airport_id = dep.airport_id
JOIN   airport arr ON f.arrival_airport_id = arr.airport_id
ORDER BY b.booking_id DESC
LIMIT 10;
```

Figure 14: Query 1 - four-table JOIN with dual airport alias

	booking_id integer	passenger text	flight_number character varying (10)	route text	seat_class character varying (20)	status character varying (20)
1	20	Oliver Smith	DG-1006	CDG > L...	Economy	Cancelled
2	19	Emily Chen	DG-1005	LHR > C...	Business	Confirmed
3	18	Mohammed Al-Ras...	DG-1004	DXB > L...	Business	Confirmed
4	17	Sarah Johnson	DG-1015	JFK > SIN	Economy	Confirmed
5	16	James Wilson	DG-1014	FRA > JFK	Business	Confirmed
6	15	Ahmed Al-Farsi	DG-1013	LHR > IST	Economy	Confirmed
7	14	Sofia Rossi	DG-1012	AMS > F...	Business	Confirmed
8	13	Ravi Sharma	DG-1011	LHR > A...	Economy	Confirmed
9	12	Emma Thompson	DG-1010	LAX > L...	Economy	Confirmed
10	11	David Kim	DG-1009	LHR > L...	First	Confirmed

Figure 15: Query 1 result - booking ID, passenger name, route, class and status

Query 2: Flight Occupancy (4-table JOIN with aggregation)

Joins flight, aircraft and airport, then left-joins booking filtered to Confirmed status. NULLIF prevents a division-by-zero error if total_seats is ever zero. The occupancy percentage is rounded to one decimal place and results are ranked from highest to lowest occupancy.

```

SELECT f.flight_number,
       dep.iata_code || ' > ' || arr.iata_code AS route,
       ac.total_seats,
       COUNT(b.booking_id) AS confirmed,
       ROUND((COUNT(b.booking_id)::DECIMAL /
              NULLIF(ac.total_seats, 0)) * 100, 1) AS occupancy_pct
FROM flight f
JOIN aircraft ac ON f.aircraft_id = ac.aircraft_id
JOIN airport dep ON f.departure_airport_id = dep.airport_id
JOIN airport arr ON f.arrival_airport_id = arr.airport_id
LEFT JOIN booking b
      ON b.flight_id = f.flight_id AND b.status = 'Confirmed'
GROUP BY f.flight_id, f.flight_number,
         dep.iata_code, arr.iata_code, ac.total_seats
ORDER BY occupancy_pct DESC NULLS LAST;

```

Figure 16: Query 2 - occupancy with NULLIF, ROUND and LEFT JOIN

	flight_number character varying (10)	route text	total_seats integer	confirmed bigint	occupancy_pct numeric
1	DG-1001	LHR > JFK	180	3	1.7
2	DG-1013	LHR > IST	124	1	0.8
3	DG-1012	AMS > F...	178	1	0.6
4	DG-1011	LHR > A...	178	1	0.6
5	DG-1007	LHR > SIN	162	1	0.6
6	DG-1002	JFK > LHR	180	1	0.6
7	DG-1003	LHR > DXB	396	2	0.5
8	DG-1005	LHR > C...	555	2	0.4
9	DG-1010	LAX > LHR	296	1	0.3
10	DG-1014	FRA > JFK	396	1	0.3
11	DG-1008	SIN > HKG	369	1	0.3
12	DG-1004	DXB > LHR	396	1	0.3
13	DG-1015	JFK > SIN	369	1	0.3
14	DG-1009	LHR > LAX	296	1	0.3
15	DG-1006	CDG > L...	555	1	0.2

Figure 17: Query 2 result - occupancy percentage per flight, ranked

Query 3: Route Revenue (4-table JOIN with SUM)

Aggregates total payment revenue by route. COALESCE returns zero rather than NULL for routes with no confirmed bookings, keeping the result set complete and sortable. Results are ordered by total revenue descending.

```

SELECT dep.iata_code || ' > ' || arr.iata_code AS route,
       COUNT(b.booking_id) AS total_bookings,
       COALESCE(SUM(pay.amount), 0) AS revenue_gbp
FROM flight f
JOIN airport dep ON f.departure_airport_id = dep.airport_id
JOIN airport arr ON f.arrival_airport_id = arr.airport_id
LEFT JOIN booking b
      ON b.flight_id = f.flight_id AND b.status = 'Confirmed'
LEFT JOIN payment pay ON pay.booking_id = b.booking_id
GROUP BY dep.iata_code, arr.iata_code
ORDER BY revenue_gbp DESC;

```

Figure 18: Query 3 - route revenue with COALESCE and SUM aggregation

	route text	total_bookings bigint	revenue_gbp numeric
1	LHR > DXB	2	2520.00
2	LHR > LAX	1	1440.00
3	LHR > JFK	3	1400.00
4	LHR > CDG	2	1278.00
5	DXB > LHR	1	860.00
6	FRA > JFK	1	780.00
7	JFK > SIN	1	610.00
8	CDG > LHR	1	570.00
9	LHR > SIN	1	550.00
10	LAX > LHR	1	495.00
11	SIN > HKG	1	480.00
12	JFK > LHR	1	360.00
13	AMS > F...	1	320.00
14	LHR > IST	1	280.00
15	LHR > A...	1	150.00

Figure 19: Query 3 result - revenue per route, highest to lowest

Query 4: Crew Workload (2-table JOIN with COUNT)

Left-joins crew and crew_assignment so that crew members with no current assignments still appear in the result with a count of zero. This gives a complete picture of crew utilisation rather than only showing those who have been assigned flights.

```
SELECT cr.first_name || ' ' || cr.last_name AS crew_member,
       cr.role,
       COUNT(ca.assignment_id) AS flights_assigned
FROM crew cr
LEFT JOIN crew_assignment ca ON ca.crew_id = cr.crew_id
GROUP BY cr.crew_id, cr.first_name, cr.last_name, cr.role
ORDER BY flights_assigned DESC;
```

Figure 20: Query 4 - crew workload with LEFT JOIN and COUNT

	crew_member text	role character varying (50)	flights_assigned bigint
1	Linda Taylor	Cabin Crew	2
2	Barbara Thomas	Cabin Crew	2
3	Charles Robins...	First Officer	2
4	Patricia Davis	First Officer	2
5	Michael Jones	Pilot	2
6	Robert Brown	Pilot	2
7	Susan White	First Officer	2
8	Joseph Harris	Cabin Crew	1
9	Richard Jackson	Pilot	1
10	Jessica Martin	Cabin Crew	1
11	William Anders...	Purser	1
12	Thomas Garcia	Pilot	1
13	Karen Martinez	Purser	1
14	Daniel Lewis	Ground Staff	0
15	Nancy Clark	Cabin Crew	0

Figure 21: Query 4 result - crew members ranked by number of flight assignments

Query 5: Full Booking History (5-table JOIN)

The most complex query in the system, joining five tables: booking, passenger, flight, airport aliased twice for departure and arrival, and payment. It returns a complete per-passenger booking history including route, seat class, booking status, payment amount, payment

method and payment status in a single result set, the kind of cross-entity view that would be impossible without relational joins.

```
SELECT p.first_name || ' ' || p.last_name AS passenger,
       f.flight_number,
       dep.iata_code || ' > ' || arr.iata_code AS route,
       b.seat_class,
       b.status,
       pay.amount,
       pay.payment_method,
       pay.status AS payment_status
FROM booking b
JOIN passenger p ON b.passenger_id = p.passenger_id
JOIN flight f ON b.flight_id = f.flight_id
JOIN airport dep ON f.departure_airport_id = dep.airport_id
JOIN airport arr ON f.arrival_airport_id = arr.airport_id
LEFT JOIN payment pay ON pay.booking_id = b.booking_id
ORDER BY p.last_name, b.booking_id;
```

Figure 22: Query 5 - five-table JOIN with full payment detail

	passenger text	flight_number character varying (10)	route text	seat_class character varying (20)	status character varying (20)	amount numeric (10,2)	payment_meth character varyi
1	Ahmed Al-Farsi	DG-1013	LHR > IST	Economy	Confirmed	280.00	PayPal
2	Mohammed Al-Ras...	DG-1003	LHR > D...	First	Confirmed	1260.00	Credit Card
3	Mohammed Al-Ras...	DG-1004	DXB > L...	Business	Confirmed	860.00	Credit Card
4	Emily Chen	DG-1002	JFK > L...	Economy	Confirmed	360.00	PayPal
5	Emily Chen	DG-1005	LHR > C...	Business	Confirmed	738.00	Bank Transfer
6	Lucas Dubois	DG-1006	CDG > L...	Business	Confirmed	570.00	Credit Card
7	Fatima Hassan	DG-1003	LHR > D...	First	Confirmed	1260.00	Credit Card
8	Sarah Johnson	DG-1001	LHR > J...	Economy	Confirmed	350.00	Debit Card
9	Sarah Johnson	DG-1015	JFK > SIN	Economy	Confirmed	610.00	Debit Card
10	David Kim	DG-1009	LHR > L...	First	Confirmed	1440.00	Bank Transfer
11	Carlos Martinez	DG-1008	SIN > H...	Economy	Confirmed	480.00	PayPal
12	Aisha Patel	DG-1001	LHR > J...	Business	Confirmed	700.00	Bank Transfer
13	Sofia Rossi	DG-1012	AMS > F...	Business	Confirmed	320.00	Credit Card
14	Ravi Sharma	DG-1011	LHR > A...	Economy	Confirmed	150.00	Debit Card
15	Oliver Smith	DG-1005	LHR > C...	First	Confirmed	540.00	Credit Card

Figure 23: Query 5 result - complete booking and payment history

2.7 Task G: CRUD Application Interface

SkyBase is implemented as a Python 3 Flask application using psycopg2 as the PostgreSQL adapter, running on localhost:5000. The application includes a login form that authenticates staff before granting access to any data. Every database query uses parameterised placeholders to prevent SQL injection. Operations that write to more than one table, such as creating a booking and its associated payment record simultaneously, use explicit transaction handling so that both writes succeed together or both are rolled back if an error occurs.

All Passengers (17)							Add Passenger	
Search by name, email or passport...							Search	
ID	NAME	EMAIL	PASSPORT	NATIONALITY	DATE OF BIRTH	ACTIONS		
#17	risa hussain	risa.hussain@email.com	CH123453	Chinese	2003-01-10	Edit Delete		
#16	Humaun Johnson	humaun.johnson@email.com	PK123456	Pakistani	2002-01-27	Edit Delete		
#15	Ahmed Al-Farsi	ahmed.alfarsi@email.com	P100015	Emirati	1975-02-28	Edit Delete		
#14	Sofia Rossi	sofia.rossi@email.com	P100014	Italian	1994-07-11	Edit Delete		
#13	Ravi Sharma	ravi.sharma@email.com	P100013	Indian	1980-03-19	Edit Delete		

Figure 24: Dashboard - entity row counts for all 10 tables and recent bookings feed

Add New Passenger

New Passenger [← Back to List](#)

FIRST NAME * LAST NAME *

EMAIL * PHONE

DATE OF BIRTH * PASSPORT NUMBER *

NATIONALITY * GENDER

[+ Add Passenger](#)

Figure 25: Passenger list with search active and per-row edit and delete controls

New Passenger Booking [← Back to Bookings](#)

PASSENGER * FLIGHT *

SEAT CLASS * PAYMENT AMOUNT (£) *

PAYMENT METHOD *

[+ Create Booking](#)

Figure 26: Add and edit passenger forms with all fields populated

Operational Reports
Live SQL-powered analytics for Diamond Group — all queries join 2+ tables

Revenue by Route

ROUTE	CONFIRMED BOOKINGS	TOTAL REVENUE (£)	REVENUE BAR
LHR → DXB	2	£2520.00	
LHR → LAX	1	£1440.00	
LHR → JFK	3	£1400.00	
LHR → CDG	2	£1278.00	
DXB → LHR	1	£860.00	
FRA → JFK	1	£780.00	
JFK → SIN	1	£610.00	

Figure 27: Add booking form and reports page with revenue, occupancy and crew data

The dashboard displays live row counts for all ten entities alongside a feed of the eight most recent bookings. The passenger module supports full CRUD: staff can search by name or email, add new records, edit existing ones, and delete a passenger record subject to referential integrity constraints. The booking form presents a passenger dropdown, a flight dropdown labelled with route and departure time, a seat class selector, and a payment section. The reports page surfaces all five SQL queries above as formatted tables, giving managers a consolidated view of route performance, crew utilisation and payment breakdowns without writing any SQL.

3. Part 2: Research and Development

3.1 Big Data

Big Data describes datasets whose volume, velocity or variety exceeds what conventional database management systems can handle within an acceptable timeframe. Laney (2001) identified the original three dimensions: volume, velocity and variety. Veracity was later added to account for data quality concerns at scale (Manyika et al., 2011).

In aviation all four dimensions apply. Volume is significant because a busy carrier generates millions of booking records, maintenance entries and sensor readings daily. Velocity matters because decisions around gate assignment, fuel loading and air traffic routing must be acted on in near real time. Variety is broad, spanning structured booking data, semi-structured maintenance logs, GPS telemetry and unstructured cabin crew reports. Veracity is arguably the most critical dimension in this sector: corrupted sensor readings can carry safety implications rather than purely operational ones.

Rolls-Royce's TotalCare programme illustrates the IoT dimension. Aircraft engines are fitted with sensors that stream live performance telemetry to ground teams throughout every flight (Rolls-Royce, 2024). A single long-haul service can produce over half a terabyte of data, which no relational database is designed to ingest in real time. This requires a distributed streaming architecture which are precisely what Big Data frameworks provide.

3.2 Hadoop and Relational Database Systems

Hadoop and relational databases are built for fundamentally different workloads. PostgreSQL manages structured, schema-fixed data with strict transactional guarantees, concurrent access control and full ACID compliance. It is the right tool for operational data such as bookings, payments and crew records where correctness and consistency are paramount. Hadoop is designed for batch processing of very large, often unstructured or semi-structured datasets distributed across a cluster of commodity hardware, where the priority is throughput rather than transactional integrity (White, 2015).

The framework has two core components. HDFS, the Hadoop Distributed File System, splits large files into fixed-size blocks and distributes them across multiple nodes in a cluster. Each block is replicated across several nodes by default, so the system can tolerate individual node failures without data loss (Shvachko et al., 2010). This replication also improves read throughput by allowing the same block to be served from the nearest available node. MapReduce is the processing model that sits on top of HDFS. A Map phase distributes portions of the input data across worker nodes and processes each chunk independently, emitting intermediate key-value pairs as output. A Reduce phase then consolidates those pairs into a final result. Running both phases in parallel across a large cluster allows petabyte-scale datasets to be processed in a fraction of the time a single machine would require.

A scenario where Hadoop clearly outperforms a relational database: a data analyst needs to identify delay patterns across ten years of national aviation records, combining structured flight logs from multiple carriers with unstructured maintenance reports and weather feeds from external providers. PostgreSQL would struggle at that combined volume and variety. Hadoop ingests all of it into HDFS and runs the analysis as a scheduled overnight batch job.

In practice the two systems coexist rather than compete: Hadoop handles historical largescale analysis while PostgreSQL manages the live operational database serving bookings and crew management in real time.

3.3 Relational, NoSQL and NewSQL Databases

Relational databases store data in fixed-schema tables and guarantee ACID properties: atomicity, consistency, isolation and durability. Their key advantages are strong data integrity, mature tooling and well-understood query semantics. The disadvantages emerge at scale: horizontal scaling across many servers is difficult, and altering a schema in a large production environment is disruptive and risky (Cattell, 2011).

NoSQL systems were built to address those specific limitations. The category covers several data models with different strengths and weaknesses. Document stores such as MongoDB offer flexible schema design as an advantage, making them well-suited to applications where records vary in structure. Their disadvantage is weaker consistency guarantees.

Columnfamily stores such as Cassandra provide very high write throughput as a clear advantage, but sacrifice relational querying capability. Graph databases such as Neo4j excel at traversing complex relationships, though they are poorly suited to tabular reporting. The common disadvantage across most NoSQL systems is relaxed consistency: they favour availability and partition tolerance over strict transactional guarantees. A passenger personalisation engine with no fixed schema suits NoSQL well; a payment ledger does not (DeCandia et al., 2007).

NewSQL systems such as CockroachDB and Google Spanner combine the horizontal scalability of NoSQL with full ACID compliance, using distributed consensus protocols to maintain consistency across geographically separated nodes. Their advantage over both relational and NoSQL systems is that they offer scale without sacrificing correctness. A globally distributed ticketing platform handling millions of concurrent seat reservations across data centres on multiple continents is a realistic NewSQL use case, where both scale and transactional integrity are required simultaneously (Pavlo and Aslett, 2016). For SkyBase at its current scale, PostgreSQL remains the right choice: the schema is well-defined, the data is structured and correctness takes priority. If Diamond Group expanded internationally across multiple regions, CockroachDB or Spanner would be worth evaluating.

4. References

Cattell, R. (2011) 'Scalable SQL and NoSQL data stores', ACM SIGMOD Record, 39(4), pp. 12-27.

- Codd, E.F. (1970) 'A relational model of data for large shared data banks', *Communications of the ACM*, 13(6), pp. 377-387.
- Date, C.J. (2004) *An Introduction to Database Systems*. 8th edn. Boston: Pearson Education.
- DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Voshall, P. and Vogels, W. (2007) 'Dynamo: Amazon's highly available key-value store', in *Proceedings of the Twenty-First ACM SIGOPS Symposium on Operating Systems Principles*. New York: ACM, pp. 205-232.
- Laney, D. (2001) *3D Data Management: Controlling Data Volume, Velocity and Variety. Application Delivery Strategies*. Stamford: META Group.
- Manyika, J., Chui, M., Brown, B., Bughin, J., Dobbs, R., Roxburgh, C. and Byers, A.H. (2011) *Big Data: The Next Frontier for Innovation, Competition, and Productivity*. McKinsey Global Institute. Available at: <https://www.mckinsey.com/businessfunctions/mckinsey-digital/our-insights/big-data-the-next-frontier-for-innovation> (Accessed: 2 April 2026).
- Pavlo, A. and Aslett, M. (2016) 'What's really new with NewSQL?', *ACM SIGMOD Record*, 45(2), pp. 45-55.
- Rolls-Royce (2024) TotalCare. Available at: <https://www.rolls-royce.com/products-and-services/civil-aerospace/services/totalcare.aspx> (Accessed: 2 April 2026).
- Shvachko, K., Kuang, H., Radia, S. and Chansler, R. (2010) 'The Hadoop distributed file system', in *Proceedings of the 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies*. Incline Village: IEEE, pp. 1-10.
- White, T. (2015) *Hadoop: The Definitive Guide*. 4th edn. Sebastopol: O'Reilly Media.